

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XII
SOLID principles



Disusun Oleh:
Intan Rhevita Arselina Suryana (105224009)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, seiring bertambahnya ukuran dan kompleksitas sebuah program, kode yang dibuat sering kali menjadi sulit dipahami, diperbaiki, dan dikembangkan kembali. Kondisi ini dapat menyebabkan meningkatnya risiko kesalahan serta waktu yang dibutuhkan untuk melakukan pemeliharaan sistem. Untuk mengatasi masalah tersebut, para pengembang menerapkan berbagai pedoman desain perangkat lunak, salah satunya adalah SOLID Principles yang diperkenalkan oleh Robert C. Martin. SOLID merupakan kumpulan lima prinsip dalam pemrograman berorientasi objek yang bertujuan membantu pengembang membuat kode yang lebih terstruktur, mudah dikelola, dan fleksibel terhadap perubahan kebutuhan di masa depan. Dengan menerapkan prinsip-prinsip ini, sebuah aplikasi dapat dikembangkan secara lebih efisien tanpa harus melakukan perubahan besar pada bagian kode yang sudah ada.

SOLID terdiri dari lima prinsip utama, yaitu Single Responsibility Principle (SRP), Open-Closed Principle (OCP), Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP), dan Dependency Inversion Principle (DIP). Masing-masing prinsip memiliki tujuan untuk mengurangi ketergantungan yang tidak perlu antar komponen serta meningkatkan kualitas desain perangkat lunak. Bagi mahasiswa yang sedang mempelajari rekayasa perangkat lunak dan pemrograman berorientasi objek, pemahaman terhadap SOLID sangat penting karena prinsip ini banyak digunakan dalam pengembangan aplikasi modern, baik skala kecil maupun besar. Selain membantu menghasilkan kode yang lebih rapi dan mudah dipahami, penerapan SOLID juga mendukung proses pengujian, pemeliharaan, dan pengembangan sistem secara berkelanjutan.

II. VARIABEL

No	Nama Methode	Return Type	Fungsi
1	Mahasiswa(nama, sks)	Constructor	Method khusus buat inisialisasi awal pas objek mahasiswa baru diciptakan.
2	getNama()	String	Jalur resmi buat ngambil atau baca data nama mahasiswa dari luar kelas.
3	getSks()	int	Jalur resmi buat ngambil atau tahu jumlah SKS mahasiswa dari luar kelas.
4	simpanData(data)	void	Kontrak abstrak di Database (sekaligus aksi cetak log di MySQL/NoSQL) buat nyimpen teks data krs.
5	hitungUKT(sks)	double	Fungsi kalkulator di UKTStrategy (dan anak-anaknya) buat ngitung biaya kuliah berdasarkan SKS.
6	tampilkanInfo()	void	Kontrak di MataKuliah buat nampilin jenis kelas (Teori, Praktikum, atau KKN) ke layar.
7	alokasiAsistenLab()	void	Aksi khusus di kelas Praktikum buat mesen asisten laboratorium yang bertugas.
8	cekPeralatanPraktikum()	void	Aksi di kelas Praktikum buat mastiin alat lab sudah siap dipakai sebelum kelas mulai.
9	validasi(mahasiswa)	boolean	Fungsi di PrasyaratValidator buat ngecek apakah sks mahasiswa mencukupi (lebih dari sama dengan 20) atau tidak.
10	cetakKRS()	void	Fungsi utilitas di PDFGenerator buat simulasi bikin file cetakan PDF lembar KRS.
11	simpanKRS(data)	void	Fungsi di KRSService yang nge-trigger database terpilih buat nge-save data KRS mahasiswa.

No	Nama Variabel	Tipe Data	Fungsi
1	nama	String	Buat nyimpen nama lengkap mahasiswa yang terdaftar.
2	sks	int	Buat nyimpen total beban SKS yang diambil sama mahasiswa.

3	data	String	Parameter isi teks informasi akademik yang mau dikirim ke database.
4	database	Database (Interface)	Atribut yang ada di KRSService buat nampung mesin database yang disuntikkan dari luar.
5	informasi	String	Variabel temporer di kelas Main buat gabungin teks nama dan sks sebelum disimpan.
6	db	Database (Interface)	Variabel objek di kelas Main buat milih tipe database yang mau dipakai aktif.
7	ukt	UKTStrategy (Interface)	Variabel objek di kelas Main buat nentuin taktik skema bayaran mahasiswa.
8	mhs	Mahasiswa	Objek referensi utama dari mahasiswa yang sedang diproses datanya.

III. DOKUMENTASI DAN PEMBAHASAN CODE

```
public class Mahasiswa {  
    private String nama;  
    private int sks;  
  
    public Mahasiswa(String nama, int sks) {  
        this.nama = nama;  
        this.sks = sks;  
    }  
  
    Windsurf: Refactor | Explain | Generate Javadoc  
    public String getNama() {  
        return nama;  
    }  
  
    Windsurf: Refactor | Explain | Generate Javadoc  
    public int getSks() {  
        return sks;  
    }  
}
```

Pada class Mahasiswa ini deklarasi atribut nama dengan tipe data String dan atribut sks dengan tipe data int menggunakan akses modifier private. Lalu deklarasi constructor yang menerima dua parameter (nama dan sks) untuk memberikan nilai awal pada objek saat pertama kali dibuat menggunakan perintah new. isi atribut nama milik *class* (this.nama) dengan nilai teks yang dikirim lewat parameter input nama, begitu juga yang this.sks = sks. Kemudian tutup blok constructor. Deklarasi metode getter bernama getNama() dengan return type String dan getSks() dengan return type int untuk mengembalikan nilai dari variable private nama dan private sks. Tutup juga kedua metode dengan “}”.

```
public interface Database {  
    void simpanData(String data);  
}
```

Deklarasi Interface publik bernama Database. Interface bertindak sebagai cetakan atau kontrak formal yang menetapkan kelakuan (behavior) apa saja yang harus dimiliki oleh class-class yang nanti mengimplementasikannya. Deklarasi juga abstract method bernama simpanData yang menerima parameter String data. Metode ini sengaja dibuat tanpa memiliki isi/blok kode ({}), dan hanya berupa cetakan

fungsi. Setiap class konkrit yang terikat kontrak dengan interface ini wajib menulis ulang dan mendefinisikan isi dari metode ini.

```
public class MySQLDatabase implements Database {
    Windsurf: Refactor | Explain | Generate Javadoc
    @Override
    public void simpanData(String data) {
        System.out.println("Data disimpan ke MySQL : " + data);
    }
}
```

Pada class public MySQLDatabase yang terikat kontrak patuh dengan interface Database dengan menggunakan kata kunci implements. Ini berarti MySQLDatabase wajib mewujudkan isi dari metode *abstract* yang ada di dalam interface Database. Lalu override sebagai tanda yang memberitahu compiler java kalau metode di bawahnya sedang melakukan fungsi abstract yang diwarisi dari interface Database. Kemudian definisikan fungsi simpanData yang menerima satu parameter string. Baris terakhir sebelum kurung kurawal kedua adalah baris perintah untuk mencetak teks konfirmasi ke layar konsol yang menyatakan bahwa data string tersebut ceritanya berhasil masuk/disimpan ke database MySQL.

```
public class NoSQLDatabase implements Database {
    Windsurf: Refactor | Explain | Generate Javadoc
    @Override
    public void simpanData(String data) {
        System.out.println("Data disimpan ke Cloud NoSQL : " + data);
    }
}
```

Pada class NoSQLDatabase yang mengimplementasikan *interface* Database (kontrak yang sama dengan MySQLDatabase sebelumnya). Ini memungkinkan program menukar jenis database dengan mudah tanpa mengubah logika utama (*Loose Coupling*). Lalu override yang menjadi Indikator bahwa metode di bawahnya mewujudkan fungsi *abstract* simpanData dari *interface* Database. Kemudian definisikan fungsi simpanData untuk variasi basis data NoSQL dan cetak teks konfirmasi ke layar bahwa data string telah berhasil disimpan ke Cloud NoSQL.

```
public interface UKTStrategy {
    double hitungUKT(int sks);
}
```

Deklarasi interface bernama UKTStrategy. Ini bertindak sebagai wadah abstrak untuk menentukan berbagai algoritma perhitungan UKT yang berbeda. Kemudian Deklarasi juga abstract method

bernama `hitungUKT` yang menerima parameter jumlah SKS (`int sks`) dan mengembalikan hasil perhitungan dalam tipe data `double` (desimal). Setiap strategi kelas wajib menjabarkan rumus hitungannya masing-masing di metode ini.

```
1 public class RegulerUKT implements UKTStrategy {
    Windsurf: Refactor | Explain | Generate Javadoc
2     @Override
3     public double hitungUKT(int sks) {
4         return sks * 100000;
5     }
6 }
```

Deklarasi class `RegulerUKT` yang mengimplementasikan *interface* `UKTStrategy`. Kelas ini bertanggung jawab atas rumus perhitungan UKT khusus mahasiswa jalur reguler. Lakukan override dan definisikan metode `hitungUKT` bawaan dari kontrak strateginya. Lalu gunakan rumus spesifik untuk kelas reguler, yaitu mengalikan jumlah SKS yang diambil dengan tarif Rp100.000 per SKS, lalu langsung mengembalikan nilainya.

```
1 public class KaryawanUKT implements UKTStrategy {
    Windsurf: Refactor | Explain | Generate Javadoc
2     @Override
3     public double hitungUKT(int sks) {
4         return sks * 150000;
5     }
6 }
```

Deklarasi class `KaryawanUKT` yang juga mengimplementasikan *interface* `UKTStrategy`. Kelas ini bertugas menangani rumus perhitungan UKT khusus untuk mahasiswa jalur kelas karyawan. Lakukan override dan definisikan metode `hitungUKT` untuk versi kelas karyawan. Gunakan juga rumus spesifik untuk kelas karyawan, yaitu mengalikan jumlah SKS dengan tarif yang lebih tinggi, yaitu Rp150.000 per SKS, lalu mengembalikan hasilnya.

```
1 public class MBKMUKT implements UKTStrategy {
    Windsurf: Refactor | Explain | Generate Javadoc
2     @Override
3     public double hitungUKT(int sks) {
4         return sks * 50000;
5     }
6 }
```

Deklarasi *class* publik bernama `MBKMUKT` yang mengimplementasikan *interface* `UKTStrategy`. Kelas ini bertugas menangani rumus perhitungan UKT khusus mahasiswa yang mengikuti program MBKM (Merdeka Belajar Kampus Merdeka).

Lakukan override dan definisikan metode hitungUKT untuk versi mahasiswa yang mengikuti program MBKM. Gunakan juga rumus perhitungan tarif khusus mahasiswa MBKM, yaitu jumlah SKS dikalikan dengan tarif Rp50.000 per SKS.

```
1 public interface MataKuliah {  
2     void tampilkanInfo();  
3 }
```

Deklarasi interface bernama MataKuliah. Ini menjadi kontrak dasar yang mendefinisikan kelakuan umum dari sebuah mata kuliah.

Deklarasi abstract method tanpa bodi bernama tampilkanInfo(). Setiap kelas konkrit yang mengimplementasikan interface ini nantinya wajib mendefinisikan cara menampilkan informasi mata kuliahnya.

```
1 public interface LabCourse {  
2     void alokasiAsistenLab();  
3     void cekPeralatanPraktikum();  
4 }
```

Deklarasi interface baru bernama LabCourse. Interface ini dibuat terpisah untuk memisahkan tanggung jawab khusus mata kuliah praktikum/laboratorium (menerapkan prinsip Interface Segregation Principle). Deklarasi abstract method untuk menentukan mekanisme alokasi atau penunjukan asisten laboratorium. Deklarasi abstract method untuk menangani prosedur pengecekan alat-alat praktikum sebelum kelas dimulai.

```
1 public class Teori implements MataKuliah {  
2     Windsurf: Refactor | Explain | Generate Javadoc  
3     @Override  
4     public void tampilkanInfo() {  
5         System.out.println(x: "Mata Kuliah Teori");  
6     }  
7 }
```

Deklarasi kelas konkrit bernama Teori yang mematuhi kontrak *interface* MataKuliah. Kelas ini merepresentasikan mata kuliah reguler yang berbasis teori di kelas saja. Proses *override* metode tampilkanInfo() agar sesuai dengan identitas dari kelas Teori. Lalu cetak teks "Mata Kuliah Teori" ke layar konsol saat metode ini dipanggil.


```

1 public class Praktikum implements MataKuliah, LabCourse {
    Windsurf: Refactor | Explain | Generate Javadoc
2     @Override
3     public void tampilkanInfo() {
4         System.out.println(x: "Mata Kuliah Praktikum");
5     }
6
    Windsurf: Refactor | Explain | Generate Javadoc
7     @Override
8     public void alokasiAsistenLab() {
9         System.out.println(x: "Mengalokasikan asisten lab");
10    }
11
    Windsurf: Refactor | Explain | Generate Javadoc
12    @Override
13    public void cekPeralatanPraktikum() {
14        System.out.println(x: "Mengecek alat praktikum");
15    }
16 }

```

Pada class Praktikum yang mengimplementasikan **dua interface sekaligus**, yaitu MataKuliah dan LabCourse. Di Java, sebuah kelas bisa mewarisi banyak kontrak kontrak interface (multiple inheritance via interface). Gunakan override dari interface MataKuliah untuk menampilkan pesan teks “Mata Kuliah Praktikum”. Lalu tambah metode pertama dari interface LabCourse untuk mence tak log alokasi asisten laboratorium. Buat juga metode kedua dari interface LabCourse untuk mencetak log pengecekan alat praktikum.

```

1 public class KKN implements MataKuliah {
    Windsurf: Refactor | Explain | Generate Javadoc
2     @Override
3     public void tampilkanInfo() {
4         System.out.println(x: "Kuliah Kerja Nyata");
5     }
6 }

```

Deklarasi kelas KKN yang mematuhi kontrak MataKuliah dan melakukan *override* fungsi tampilkanInfo() untuk mencetak tulisan "Kuliah Kerja Nyata".

```

1 public class PrasyaratValidator {
    Windsurf: Refactor | Explain | Generate Javadoc
2     public boolean validasi(Mahasiswa mahasiswa) {
3         return mahasiswa.getSks() >= 20;
4     }
5 }

```

Deklarasi kelas validator dengan fungsi validasi yang menerima objek Mahasiswa dan mengembalikan nilai boolean (true/false). Lalu periksa apakah jumlah SKS mahasiswa tersebut (lewat mahasiswa.getSks()) sudah mencapai angka 20 atau lebih. Hasil evaluasi kondisi perbandingan ini langsung dikembalikan.

```

1 public class PDFGenerator {
    Windsurf: Refactor | Explain | Generate Javadoc
2     public void cetakKRS() {
3         System.out.println(x: "Mencetak PDF KRS");
4     }
5 }

```

Kelas utilitas sederhana yang memiliki fungsi cetakKRS() untuk mensimulasikan pencetakan lembar KRS mahasiswa ke dalam format dokumen PDF.

```

1 public class KRSService {
2     private Database database;
3     public KRSService(Database database) {
4         this.database = database;
5     }
6
7     Windsurf: Refactor | Explain | Generate Javadoc
8     public void simpanKRS(String data) {
9         database.simpanData(data);
10    }

```

Kelas layanan KRS yang memiliki atribut objek bertipe interface Database. Pada bagian constructor Pola ini disebut Dependency Injection lewat constructor. Kelas ini tidak membuat sendiri objek databasenya (tidak pakai new MySQLDatabase()), melainkan menerima database jenis apapun dari luar dan menyimpannya ke atribut this.database. lalu tambahkan method simpanKRS untuk memproses penyimpanan KRS. Di dalamnya, program memanggil fungsi .simpanData(data) milik objek database yang disuntikkan tadi.

```

1 public class Main {
    Run | Debug | Windsurf: Refactor | Explain | Generate Javadoc
2     public static void main(String[] args) {
3         Mahasiswa mhs = new Mahasiswa(nama: "Russel", sks: 22);
4         UKTStrategy ukt = new MBKMUKT();
5         System.out.println("UKT : " + ukt.hitungUKT(mhs.getSks()));
6     };
7
8     Database db = new NoSQLDatabase();
9     KRSService service = new KRSService(db);
10    service.simpanKRS(data: "KRS Semester 4");
11
12    MataKuliah teori = new Teori();
13    teori.tampilkanInfo();
14    MataKuliah praktikum = new Praktikum();
15    praktikum.tampilkanInfo();
16 }
17 }

```

Pada public class Main ini, baris pertama dan kedua adalah Titik awal (*entry point*) jalannya seluruh eksekusi program Java di dalam kelas

Main. Lalu Instansiasi objek Mahasiswa baru bernama "Russel" dengan jumlah mengambil beban 22 SKS. Lakukan polimorfisme strategi, membuat objek dengan strategi khusus mahasiswa jalur MBKMUKT. Kemudian hitung UKT milik "Rin" dengan mengirimkan SKS miliknya (20) ke dalam metode hitungUKT taktik MBKM (20 X 50.000), lalu mencetak hasilnya ke layar. Setelah itu siapkan objek mesin database jenis NoSQLDatabase. Inject objek db tersebut ke dalam constructor KRSService dan panggil metode simpanKRS. Karena di baris 9 kita menyuntikkan NoSQLDatabase, maka kalimat "KRS Semester 5" otomatis akan diproses dan disimpan menggunakan mesin NoSQL. Selanjutnya buat objek berwujud kelas Teori namun ditampung menggunakan variabel bertipe *interface* MataKuliah, kemudian mencetak informasi kelas teori tersebut. Buat juga objek berwujud kelas Praktikum yang juga ditampung di variabel bertipe MataKuliah (Polimorfisme), lalu memanggil fungsi cetak informasinya. Terakhir tutup semua program dengan kurung kurawal “}”.

IV. KESIMPULAN

Kesimpulan yang bisa saya ambil dari kelanjutan praktikum kali ini adalah saya jadi jauh lebih memahami bagaimana cara menyusun arsitektur kode program yang rapi, fleksibel, dan saling terhubung lewat konsep Object-Oriented Programming (OOP) tingkat lanjut. Melalui pembuatan program sistem akademik ini, saya tidak hanya membuat cetakan objek dasar seperti pada class Mahasiswa, tetapi juga berhasil mengimplementasikan hubungan kontrak antar-elemen menggunakan interface. Saya jadi mengerti bahwa dengan memanfaatkan interface seperti Database dan UKTStrategy, kita bisa membuat program yang dinamis karena jenis database (MySQL/NoSQL) atau strategi hitungan biaya kuliah (Reguler, Karyawan, MBKM) bisa ditukar-tukar dengan sangat mudah tanpa merusak kodingan utama yang sudah berjalan.

Selain itu, praktikum ini memberikan gambaran nyata kepada saya tentang bagaimana sebuah sistem perangkat lunak di dunia kerja dirancang dengan memisahkan fungsi-fungsi secara spesifik. Saya bisa langsung mempraktikkan pembuatan kelas validator khusus pada PrasyaratValidator untuk mengecek kelayakan SKS mahasiswa, kelas generator pada PDFGenerator untuk mencetak dokumen, hingga penerapan Dependency Injection pada KRSService. Melalui gabungan konsep polimorfisme, loose coupling, dan penggunaan banyak interface sekaligus (multiple interfaces) pada kelas Praktikum, saya merasa logika pemrograman saya jadi lebih terasah dalam membangun aplikasi yang modular, mudah dirawat, dan siap untuk dikembangkan ke skala yang lebih besar di kemudian hari.

V. DAFTAR PUSTAKA

https://principles.design/examples/solid-design-principles?utm_source

https://www.techtarget.com/searcharchitecture/feature/An-intro-to-the-5-SOLID-principles-of-object-oriented-design?utm_source